

ActInf GuestStream 095.1 ~ Rithvik Prakki: Active Inference for Self-Organizing Multi-LLM Systems

GuestStream | Rithvik Prakki: Active Inference for Self-Organizing Multi-LLM Systems | 56:50

Hello and welcome everyone. It's January 16th, 2025. We're in Active Inference Guest Stream number 95.1 with Ritvik Praki discussing Active Inference for Self-Organizing Multi-LLM Systems, a Bayesian Thermodynamic Approach to Adaptation. So Ritvik, thank you for joining. Looking forward to your presentation and sharing more and discussing. So thank you to you. Great. Thank you. Thank you, Daniel, for having me. Yeah. So I'm going to talk about kind of Active Inference for self-organizing multi-LLM systems. So basically making LLM agents better with Active Inference. So to motivate the problem a little bit, current multi-LLM agents kind of have a lot of complex parameters. For example, prompts, agent architecture, tool use. Like these are things that you can't just kind of fix with hyperparameter optimization. You have to find better, more intelligent ways to improve these in time. And that's not really a thing right now. Also, another thing is that LLMs themselves are used as a vector adaptation for the agent in many ways. So for example, in the Voyager paper, LLMs are used to acquire new skills and then use those skills to explore the environment of Minecraft further, which is useful, but also quite limited. And then the third thing is that there's no formalized mechanism of balancing the exploration and exploitation trade-off. Right now, it's kind of just using LLMs internal abilities to go about that, but there's no formalized mechanism. And that kind of limits a lot of the potential. Now, our solution is using Active Inference to kind of overcome a lot of these problems. So the first thing is that the best possible prompt for different LLMs within the agent can be learned over time. These things are something that we can figure out from the environment. We can do search to figure out what are the best new prompting techniques. So we can test out different prompts to see which ones give better scores on certain metrics. So these things can be learned over time. And the Active Inference agent can help with that because now the Active Inference agent becomes the vector adaptation, allowing for real learning through belief updates and structural updates versus just solely in-context adaptation, which is what LLMs can do. So we're actually learning and we're actually adding information and updating beliefs within our model. And Active Inference acts as the formalized mechanism to balance the explore-explore boundary, allowing for a more principled understanding of uncertainty, and also giving us the option to verify optimality. See if the path of selecting actions is actually an optimal one or not, which we can't do with LLMs directly. So Active Inference works through a generative model. And the generative model we typically use for active inference in discrete time is called the Partially Observable Markov Decision Process, POMDP. Now it's Markov because current states are only influenced directly by past states and it's partially observable as opposed to a regular MDP because the agent doesn't have the ability to directly observe the states. It must go through an observation, which allows it to update beliefs about the states without any direct observation of the states. So within Active Inference, we have several matrices that we put into these POMDPs to encode the actual information that we're learning from the environment or that is contained within the environment. So the first thing is the A matrix

or the likelihood matrix. This is basically just showing the relationships between the likelihood, showing the relationships between states and observations. So what are the chances that this observation is related to this state or this state produces this type of observation?

The B matrix is showing the relationship between states at T plus one and states at T, time T. So basically, what is the probability that this state will occur at this time, given that this state occurred at a past time? Now the C matrix encodes the preferences at the active inference agent. So for example, if some of the states that this model had was sunny weather and rainy weather, the C matrix would encode that the model likes sunny weather. And then the D matrix encodes the prior beliefs of the states. Now the ultimate goal is to minimize surprise. Because low surprise means that the model we have is actually a good representation of its environment. But if we don't, if we have high surprise, that means we're not really understanding the environment well, because every observation we're getting is surprising us. So the goal is to minimize surprise. And however, the issue with just directly minimizing surprise, which is shown in this equation here as negative log of the probability over all observations, is that this is typically intractable when the state space gets really big, and it grows exponentially. So we can't directly minimize the surprise. So we have to turn that into a new quantity that we can more easily approximate to instead of directly computing. So this is how we derive variational free energy, by doing a variety of steps from Bayes' theorem to get to this final equation. Now the variational free energy equation is constituted of two terms, the complexity term and the accuracy term. So the complexity term is basically just showing us the difference between the distributions for the prior beliefs and the posterior beliefs. And the accuracy term is just trying to show us how close, how well the observations that, how well the states that we're seeing, and how well those observations align with the states that we're seeing, or the states that we believe in. So effectively, the goal is to minimize the difference between our prior beliefs and posterior beliefs, and also to have our beliefs be as accurate as possible given the observations. Now the way this actually happens is a minimization process. Like we said before, this is a quantity we want to approximate to. So this equation right here, the $\frac{df}{dq_s}$, is just the partial derivative of free energy in terms of the posterior beliefs. This is, this first equation is just showing the, showing it in terms for one state factor. And when we do this, we want the derivative to equal zero over time. But in practice, what happens is the max, we have a max number of iterations. Once those max number of iterations are reached, or the change in variational free energy falls below a certain threshold, the optimization process stops, and we have our final posterior beliefs that have minimized the quantities we want to minimize. This is the final posterior belief equation shown right here, basically just showing how the final posterior beliefs will look, how they're influenced by the prior beliefs, and how they're influenced by the observations, the current state factor, and also the other state factors. Now the second quantity in active inference that we're trying to minimize is the expected free energy. Now this is related to policies and actions and future states, as opposed to variational free energy, which is very in the present. So expected free energy is made up of two terms,

information gain and pragmatic value. So information gain is just trying to show how much we expect to gain by doing a certain action. So the way that transitions are determined is they're conditioned on an action. So if we do this action, what is the likely outcome? So right here we're saying if we do this specific action, so a policy, if we execute the certain policy, what is the likelihood of having significant information gain? If we have a lot of information gain, that means it's a useful action because it's highly informative and that's what we want to optimize for. So the second term is called pragmatic value, also known as expected utility. This term is useful for comparing the expected observations to our preferred observations. So as I said before, the C matrix is the one that encodes the preferred observations. And we want to, since we know, since we have some idea of what the expected states are from the B matrix, we also have some idea of what the expected states are in the A matrix, we can figure out what the expected observation should be as well. And on those expected observations, we can do a computation with the C matrix to see how high the expected utility or the expected level of preferred observations exists in the action we're about to commit.

The final part of the final part we need for this paper, at least, is the A matrix updates. So the A matrix is the, like I said before, the matrix that encodes the relationship between states and observations. But in our specific implementation, the agent doesn't initially know exactly how the states mapped observations. It's intended to learn that over time by getting in new observations.

So to do that, we need to have some mechanism to update it. And this is the mechanism. So we do a similar thing we did with variational free energy, where we take the derivative of variational free energy with respect to the modality at, or like the A matrix with a certain modality. And we basically compute that over time.

And then we take this equation, and then this equation turns into the final derived equation below. This final derived equation below is just explaining how we should update the A matrix. So the main updates are going to come from the outer product between the observation and the current belief. So for example, if you get enough, if you have two states that are sunny and rainy, and the observation that you get is that it's sunny, all we would do is update the, we would update the aspect of the sunny, the sunny part of the A matrix to say that sunny, to say that we're getting an observation that's sunny, and then add those to the counts of the A matrix. So getting to the actual specific work done.

So this whole generative model consists of three state factors. So the first is prompt combinations. The second is search states or search terms. And the third is the info state.

And then there's seven observation modalities. So the accuracy, relevance, and comprehensiveness observation modalities are directly tied to the prompt states. The information relevance, information usefulness, and source quality modalities are directly related to the search state. And then the info state just has three modalities with an info state observation modality and an info state state factor. So they're directly related to each other. So that means any observation observation, and the info state modality is just a direct reflection of the state factor.

So this just kind of shows, this figure just kind of shows the A matrices for each of these state

factors and their relationship with the observation modality. So the first three is just for the prompt quality matrix. So this is basically just showing the relationship between the three quality metrics we described, the accuracy, relevance, and comprehensiveness, and the prompt states. Now initially, you see all these squares are just blue. This is because the model has engaged in no learning. It's starting with a complete blank slate and has no understanding of the environment in this specific case. Now, the way the generative process actually works is that agents can select actions. So they can either select prompt actions or search actions like we showed before. These are the two controllable states. Now what a prompt action will do is it'll take a certain set of prompts that the agent selects, it'll run them through the actual research agent that we're trying to optimize here. It'll run them through an agent, it'll give certain observations. So as you can see this kind of JSON structure here, what we're doing is we're using a chat to T4O mini. And that's just going to give us a score from 0.0 to 1.0, counting up by 0.1 on the three different metrics we need. So for example, if an agent selects a prompt action, we first run that prompt, run those set of prompts through the agent, we get a final result. On that final result, Chesh T4O mini will evaluate it and then give us three scores. Those three scores will then get turned into an observation vector. And that observation vector will get passed back to the active inference agent to use as observation. Now this is the exact same process that happens for the search action as well, just with a different prompt. So this is actually the prompt for the search action. As you can see, we're telling Chesh T4O to return scores on info relevance, usefulness, and source quality. Now the info state is unique because it's determined purely by the amount of knowledge base. So the way that knowledge base works is by the amount as search actions are carried out and more information is added to the knowledge base. The info state goes from no knowledge to some knowledge to a high level of knowledge. And those are observations that are directly getting passed from the environment to the agent. So the agent doesn't actually have any encoding of the information it's getting from the search actions. All it knows is that there's some knowledge state coming from the environment and it's using that as a way to determine what to do next and what observations it prefers. So yeah, that's where that's where that's coming from. Now for the results, this is the same figure I showed before, but now with learned environment mappings. So as you can see for the first three matrices, these are the prompts, the agent for each prompt has a distribution over what scores of expecting. So for example, for the first prompt, it's going for eight and nine. So it has a distribution saying it's most likely expecting. So this is the accuracy. This is the accuracy metric. So saying that for prompt one, for prompt combination one, it's generally going to expect an eight or nine on the accuracy metric. And it does that all the way through for all the prompts. And it does the same thing for search as well. So this is the info relevance metric. So for example, it's saying that for prompt one, it's generally going to expect a score of nine out of 10, or point nine out of 1.0 on the info relevance metric. So kind of all that and then the info state metric is just concentrated around the second observation, the second observation is just high info. So like detailed information from the environment. This is because the agent was running

over

20 trials. And once the info state metric gets to the knowledge base that we have gets to a high level pretty quickly, and then so that the info state kind of just keeps getting observations about that it's in a high level of information has high level of information and she keeps getting those observations. So it's all concentrated around that observation specifically. Now, one of the main points of this was to show how we could show how we could go through the explore exploit trade off really well by using active inference. And this is kind of showing that so this is some time series data from time step zero all the way to time step 80. The red dots are search actions, and the blue dots are prompt actions. So as you can see, we have a preponderance of search actions in the initial stages because the agent is trying to learn the environment, trying to get more information and stuff like that. So it's trying to kind of understand how prompts work, how prompt prompting works. And then at the towards the end is there's mainly search actions or sorry, mainly prompt actions as the agent is testing these prompts to see if they work well, or if they work don't work well, kind of using the information it's gotten from the search actions. Now we know it's not actually using this is more kind of a simulation.

But it's using it in the sense that it knows that we already have a lot of information from the environment because of that info state observation it's getting.

So in conclusion, what we what we did here was that we had the model effectively learn the environment's generative process, mapping the hidden mapping the observations to the hidden states. And we were able to simulate human like exploratory behavior with active searching and testing multiple possibilities. And then the main thing is that we were able to enhance the capability of LLMs as well as enhanced capability of active inference, because we gave LLMs a formalized structure that they didn't have before to execute learning through. And we gave active inference the ability to operate in high dimensional spaces by integrating LLMs. What if we had to try to encode this functionality directly into the active inference agent, it would have been much more harder and much more complex than just having the LLM outsource everything to the LLM and have it do a lot of the work.

So we kind of had improvements on both ends through the system.

Now I just want to talk about some

some future directions for this. So obviously, the one big thing that this model doesn't have is encoding actual knowledge from the environment. So as we perform search actions, we're getting information about prompts, right? So we're saying, so if we look up, if we look up, what are the best ways to prompt for ChatGPT? It'll say like tree of thought, chain of thought, blah, blah, blah, et cetera, et cetera. But none of this information is actually getting encoded into the agent. All that's happening is that there's an info state, right? And it's using that info state to determine future actions. But it would be much more useful if the active inference agent actually knew why search certain prompts or certain prompt combinations were good based on the search actions, and then use that information directly to influence those actions. So making that that would require some sort of hierarchical model and making that would be a good future direction for this. Another thing is giving the ability, giving the agent the ability to dynamically expand its state space to not be constrained to a fixed set of prompts. So here, what we

did was we just had a fixed set of prompts for prompts for each part of the agent. And obviously, like, there's much more the space of possible prompts is infinite space. And there's, there's a lot more we could explore if we weren't constrained by a fixed state space. So expanding on that would be very useful. And also, the third thing would be to equip the agent with the ability to modify other complex parameters, such as instead of just prompts, maybe also be able to modify the architecture of the agent, or the tools it's using, and stuff like that. The initial goal is to make a general kind of agent brain that has the capability to plug be plugged into any LLM agent and exhibit human levels of learning, modification, structure learning, and stuff like that. So yeah, I think that's it.

Awesome. Thank you for the presentation. For those watching live, they can write questions while I'm just cropping and getting everything back. Maybe share a little bit, how did you come to this project?

Where are you at in your education and research and all of this?

Yeah, sure. So yeah, my, well, I only found out about active inference, maybe a year ago, and kind of how I happened upon it is that I wanted to kind of do this general idea that I just said, like the agent brain. So because I noticed that LLMs were pretty static, and they're like LLM agents were pretty static, and then humans had to directly go in and like, Oh, let me fix this prompt, or let me, let me change the order of these two nodes and stuff like that. And the same kind of, you know, LLMs, LLMs themselves can

do a lot of that improvement work. But it seemed like there was a big hole there that we could just fill by having some sort of self organizing behavior. And that's kind of how it happened upon active inference, because it was this like model based. So we were responsible for figuring out how to like actually structure the generative model and the generative process that the agent was going to work with. And it was very sample efficient, which I really liked because there's generally don't get that many samples in a typical workload. So for example, if we're using an agent in production, we wouldn't really be getting unless it was being used that massive scale, we wouldn't be getting too many examples that we could work with to improve certain functions. And the sample efficiency of active inference really kind of helps with that issue. And then the other thing was the reinforcement learning, obviously, just having some aspect of being able to learn over time.

With LLMs, you can't really do that. It's all in context. And I'm suspicious that in context learning is or just having information in context is enough to replicate the abilities that a reinforcement learning scheme can provide. So that's kind of how I happened upon this and started looking into it. I actually read the Thomas Parr, Karl Friston textbook on this. And that's kind of how I got into it started doing some work. And yeah, kind of came to this.

Cool. Okay, I'm gonna read some questions from Arun Naranjan, a colleague at the Institute, and then any other questions in the live chat, we'll see. Okay, so thanks. Thanks, Arun for the awesome submitted questions. All right. What does it mean to ask an LLM for a numerical score of something? Can we really convert the sampling of the next token's probability distribution of an LLM into a meaningful quantitative observation?

Hmm. So

It's kind of a, the LLM observation is kind of a, maybe like approximation or like, kind of a, maybe an easy way to do this, basically. So all we're doing is, we're saying like, because the best, the best text readers at this time in AI, or text understanding machines kind of are LLMs. Because the way this whole, the way this is working is that the LLMs are getting just a bunch of text, because the research agent is outputting a bunch of text. And the active inference agent directly can't really do much with this, because language is a hard space to understand. And active inference agents would need a lot more improvements to be able to understand that space. So basically, all we're doing is that we're kind of simplifying the complex state space of just the English language, and we're simplifying it into something the active inference agent can understand. So what we're doing is that the LLM can understand language, and it can kind of generate certain scores. Obviously, the LLM scores aren't as useful or nuanced as a human scoring certain like text would be. So for example, if there's an essay competition, an LLM would probably be a much worse score for this, for those essays in a consistent manner than a human would be. Because it's kind of hard to determine what the standards are. But for something like this, where the text is relatively simple, it's pretty easy for the LLM to just say, this is the score I'm giving to the text, we give it certain like guidelines to follow when assigning scores. So it just gives a certain score, and that score is kind of a, basically a simplification of the state space of all possible language into just three values that we can use to kind of help the active inference agent learn. Now, ideally, it would just be the active inference agent directly digesting the language. But this is kind of an easy way to overcome that massive gap in complexity.

Yeah, great response. I'll add a few comments. You pointed right there that a longer term research agenda is like an

active inference linguistics model using some kind of syntax and semantics on the natural language itself. And so

some kind of, you know, mega hierarchical model or something cool like that. But in the meanwhile, for specific purposes, we can take natural language expressions, use LLMs, especially with the JSON or structured output, and project them onto numerical lower dimensional state spaces, like how many bird species are mentioned in this paragraph, only respond with a specific number, three.

And then you have an A matrix that takes in as an observation, the number of bird species observed, and then maps to

some other variable for the ecosystem, or how many facts in this paragraph correspond to this circumstance. And so then the active inference generative model can pick up at the interface of the output of the structured LLM return.

So it's a design pattern, I think that's very rapidly emerging, and it's exciting to see that you're working on it, because it really does bring together this mega parametric, you know, millions to billions of parameters, high dimensional state spaces, more generic architectures for LLM and such.

And then take the output, sometimes crystallized down to just like a single scalar value, or to a vector of numerical values,

pick up there with the more interpretable smaller and sample efficiency, and direct belief updating.

And then just one more piece I'll say there's, you can get an LLM to give a quantitative or natural linguistic representation of its own uncertainty.

But then there's this kind of head scratching moment where it's like, well, even if it says that it's 55% confident,

that is the most likely chain of tokens for it to say.

It's not a true introspection of its actual uncertainty.

And so again, further down the road, that's a promising avenue for active inference models that can introspect and accurately have their own uncertainties.

However, in the meanwhile, this is one way that we can make the best of what exists today and what we can build tomorrow.

Exactly. And one other thing I'll add is that some kind of what I'm trying to do as well is

LLMs can already do a lot. So kind of have LLMs do as much as possible, but also try to

kind of like what I was saying before, like having the active inference nature be able to encode the information itself from the environment.

Like add things to the active inference agent over time and then offset some of the responsibility of the LLM so that we're getting the actual interpretability of like proper uncertainty estimation and all that stuff within the active inference agent without having to rely on the LLM.

But we kind of can like, basically like if active inference is here and the LLM is doing all this, we can kind of just like, you know, move that gap over time as we develop more.

But we're still getting the performance to start with.

You know, that's kind of funny. It makes me think of like a grass is always greener or some kind of compliment where for the LLMs, we might gain interpretability by projecting down to smaller states based models that have real quantitative variance estimators.

Whereas for active inference models, we can gain interpretability and accessibility by translating them out to natural language.

So, you know, yeah, yeah, that's a great way to put it. Yeah.

More tools better. Okay. Arun asks the next question.

Over what timescales did your active inference plus LLM agent execute?

How did that change with the complexity of the questions asked?

What was the time, the runtime dynamics?

Like how long does one full loop take?

Yeah.

I mean, it's pretty short.

So it was 20.

It was, it was about like 80, 80 time steps.

So kind of just 80, like 20 trials, four time steps for trial.

So the time steps only were relevant to the info state, the info state or information state, because the other two didn't really have any transition dynamics.

But effectively, yeah, what it would just be is that for each time step, there'd be some sort of action.

The LLM would run through all the stuff it had to run through for the agent.

So the research agent would run that we're trying to optimize.

And then the evaluation on that result would run.

And then we would have that observation.

We'd put it back into the active inference model.

So that would be one full time step.

So that maybe took like in terms of actual time to run the research agent, run the evaluation, and then the active inference model itself.

Maybe each time step took like 10 seconds.

It was relatively quick, especially because we're using like 4.0 mini, which is like faster than something like 4.0 maybe.

And the active inference agent was like the state space was pretty small.

So it didn't take much time for the active inference agent to run as well.

So, yeah.

Cool.

Yeah, I could see the latency of the LLM being a major determinant, whether you use kind of a smaller or a faster call or whether you do something that takes a bit to get chugging.

Okay, cool.

All right.

Arun's next question.

Did you actually get good answers to research questions you asked?

Well, the LLM itself already worked.

So, or the research agent itself already worked.

So like it had certain prompts and it would output something.

So, for example, if I asked like, give me like a report on coffee or something.

It'll go through the entire, you know, it was basically five aspects of the research agent.

It was planning, reviewing, actually like, or getting information from the internet, answering the question.

And if there's like any issues with the answer, then it would route back to the initial planning stage and then kind of go back from there.

So, the agent itself was already like relatively good at giving answers, giving good answers or giving good reports on whatever question you asked.

And mainly the point of this was just to show that, because like if you think about it in time, like the prompts will become like worse and worse every time.

Because prompting techniques get better.

Well, they'll get worse and worse in the sense that like we, our knowledge of how to prompt increases, for example.

Like we initially used to only think about like, oh, like step-by-step prompting.

That was like the max of what prompting could be.

Now we have like so many different techniques.

Like we have like tree of thought, chain of thought, react framework, blah, blah, blah.

There's like so many different techniques.

And if we had started in like 2023 with just the step-by-step prompting and we as a human didn't go back in there and fix it ourselves,

then we would just be stuck with those bad prompts compared to where the literature is at right now.

So the idea was that, well, for this was a fixed state space with just like, it was like five prompts for agent that we had to work with.

And then those would be formulated into combinations of prompts that the active inference agent could select.

So it's just kind of a, like a simple way to show that like the active inference agent could pick these prompts, pick the best one, research from the environment,

and then kind of do this whole process to try to learn like, which is the best prompt in that very small state space over time.

So it wasn't like, it's not going to change the effectiveness of the research agent too much because the state space of the prompts is so small.

But I mean, it was already good to begin with and it was good to end with.

Yeah.

Thanks.

A few comments on that.

We're seeing at this moment, just such an exponential increase in the capacity.

So as you point out, even six months or a year can be very different.

And it's an exciting time for understanding research and cognitive science agents overall.

Like I think about the graph structured knowledge bases, like Bueller's graph prefixor type models where, where, you know, limitations of in context learning that you pointed to can be addressed with some operations on knowledge graphs.

And it's like, we're kind of exploring motifs and initial pattern designs and proofs of concepts for how these kinds of synthetic intelligence systems and augmented systems with human and non-human entities can be collaborating in information spaces epistemically.

And like everything from the user interface and the accessibility and the human computer side on through what are we really yielding?

Are we synthesizing new molecules?

Are these revealing errors in previous derivations in papers?

Are these, you know, shorter algorithms?

Like, so it's just like, wow, happy 2025, huh?

Right.

Right.

Um, how, uh, Arun asked, how did this compare to purely LLM driven agents?

For example, LLM plus while loop plus tool calling.

Yeah.

So using LLMs, um, we can definitely use like LLMs directly to improve the agents.

So kind of just say like, so we have this research agent, we just have the LLM go in and say, um, like if every time, like every, like let's say like every day do research on like all these topics around prompting, um, synthesize that research, put it in your context and then use that context to like, look at all these five prompts, make them better.

Now, if we just said that the LLM is going to do way better than the active inference model right now, because obviously LLM can explore an infinite state space of language.

And we only have like five prompts we're looking at and we can't really like we're right now.

There's no actual thing where like the search doing the search is actually changing the prompts or changing the prompt selection mechanism or anything like that.

So yeah, LLM by itself would perform way better than the model currently.

It's more like, but the issue is that like at the end of the day, it's all in context learning with LLMs and there's no way to maintain like a real understanding of the environment over time.

Like actually encoded within belief updates and these matrices and that's what the active inference is bringing.

And this is more kind of like a proof of concept to show that like we could do that at some point because just thinking about it theoretically, an easier way to improve this is just to say kind of like how I said, like the LLM could just take in context, blah, blah, blah.

Now we could just do something simpler where we could say, now the LLM go do this research every day, take in that, put it in your context.

Now turn that into states that we can, or state factors that we can put within an active inference agent.

Now the active inference agent has a whole new set of state factors and states within those factors, and now it can use that directly to learn about new prompting techniques.

And it has new, like it's dynamic, state space is dynamically going to change because the LLM is adding new prompts all the time.

And this way, like it's a much better way of having the active inference agent actually learn these prompts.

And it's probably, I mean, we don't know yet, but most likely it's going to perform better than the LLM just doing it in context by itself.

But it's like a pretty simple way because of just like having the LLM do a lot of the work.

Because without that, the active inference agent, like the same kind of process I described, just purely within active inference would take a really long time to figure out.

Yeah, that reminds me of how many ways we're seeing pop up that computer science and AI is sort of rediscovering or reinventing cognitive science.

Like knowing that versus knowing how, knowing where to look to get more information.

And then it's like, okay, well, let's just say we are going to have our grad student, I

mean research agent, and they're going to be doing some daily task.

We say, well, I want the report to be balancing accuracy and complexity.

Well, that's variational free energy.

Or in terms of prospective experimental design, what should be the criterion?

Well, the reinforcement strategy is with some sort of large or curated or labeled database, train an auxiliary reward function, and then just go ahead and try to hill climb on that reward function.

Well, what if we're into the unknown?

We're on the frontier.

We don't necessarily have a training data set.

Or maybe we wouldn't even want one, even if we could.

So then what criteria should we use that's directly based on the model?

That yields pragmatic and epistemic value in the expected free energy, this explicit explore, exploit trade-off.

And the ability to modulate between returning expected results with the high expected utility to learning.

And then it's like, just like you gave that kind of daily duty cycle with like every day go and research something.

And then that could be encoded kind of during sleep into beliefs.

Like what is the zero to one and a variance on what is the likelihood that this is going to happen?

Or what's the efficacy of this treatment over that treatment?

And then that gives an actual information gain term so that it doesn't just continue to do the same internet searches again and again.

Rather, those searches are tuned for epistemic value.

So it's like, and then even to the chain of thought and tree of thought, well, this is where we get into questions like rumination, where you could have at a hyperparametric level, you could think too little and leave good thoughts kind of unthought on the table, so to speak. Or you could ruminate and you could think too long and be past the point of diminishing return.

Well, where is the Pareto optimal point for how long you should be thinking?

That's a metacognition question.

That's a cognitive security question.

And so it's like, it's really interesting to see, even though they've been coming from the horizon with these mega parametric models, they're actually converging to some of the formalisms and the trade-offs that cognitive science has been working with for a long time related to everything from like epistemic foraging and memory and skill to metacognition.

So, I mean, so cool how it's all coming together.

Right.

I think like Google even recently was focusing one of their models on Surprise.

So they were using Surprise as the reward function to kind of improve their models.

And it's kind of like, that's, I mean, that's active inference right there.

So it's kind of, it is kind of cool that we're seeing these concepts and active inference kind of extend out to just like LLMs and diffusion models and stuff like that.

Yeah.

Yeah.

And, and although minimizing Surprise is maximizing model evidence, there isn't a tractable heuristic to bound success or evidence.

Whereas there is one to bound Surprise, again, even though they're kind of dual.

So there's an interesting relationship there.

Sure.

Okay.

Next Arun question.

Why does the A matrix need a specific action dependence?

I believe PyMDP should create a B control state for every action anyway.

So you should be able to set those out of the box as it were.

Well, the A matrix doesn't have any action dependence.

It's just the, the actions are able to provide observations that the A matrix can use to update itself.

But the A matrix itself doesn't have any like action dependence.

It is all through the B matrix.

So the one thing is that like the B matrix, like the prompt and search states are controllable, but they don't have any real, like any like legitimate transition dynamics.

So like, just because like, if we commit some sort of action, it's not actually causing the prompt combination at one state to transition to the prompt combination at another state.

It's more that we are picking those prompt combinations independently or independently in the sense that like, it's basically just going to be deriving from expected free energy.

So whichever is the lowest expected free energy is the one we're picking.

But there's no, like the action isn't directly causing the transition from prompt combination one to prompt combination two.

The only thing where that's actually happening is the info state.

So like, I think in the code or in the code, there's like B matrix learning as well, but it really only happens for the info state.

The other, the B matrices for the prompt states and the search states are completely the same over time.

Kind of showing that like, they really don't have any direct, this action causes this state

to this state kind of, you know, dynamics there.

Cool.

Thanks for clarifying.

And this is sort of a classic design degree of freedom or openness for those who have explored these cognitive models, which is what exactly should the agents know about the consequences of its action?

Because if it doesn't know the consequences of action, it's just going to be flailing wildly.

Whereas if it's given too much information about the consequences of action, it's kind of like putting the cat in the hat because it's going to know exactly what it's going to do.

So that's where having the B matrix learning and having these learning methods is really cool and really helpful because then you can see, okay, with what learning rate and what kind of variability in action and in the generative process in the niche, what dynamics yield over what time scales, the convergence to veridical learning of the consequences of action.

So maybe it needs to spin 10,000 prompts out before understands the consequences of certain kinds of prompting.

Maybe it only needs to throw out a few depending on what it is.

So, okay.

And then one more question from Arun.

And then if anyone in the live chat has questions, they can write it.

He wrote similar question to our Bellamy, another discord colleague.

How did you find using land graph?

Land graph.

It seems like quite a polarizing framework, but still probably easier to use than PyMDP sweating emoji.

Yeah, definitely easier to use than PyMDP.

I had to do a lot of like going into PyMDP and changing like a lot of those like source code kind of to like make all of this work, which was useful for me, but it was, but yeah, land graph.

Actually, I, I've run a company where we build AI agents for other clients and companies like that.

And we typically use land graph for everything.

So it wasn't too hard because I'm kind of, I have a lot of experience in that, but yeah, but the PyMDP was definitely harder.

That's for sure.

What, what, what functionality does the land graph provide?

Like how does it, and how does it interface or pass to PyMDP?

Yeah.

Yeah.

Yeah.

So, um, the land graph is basically just a framework to kind of create nodes and edges.

And the main other thing is that it can save state between like agent runs.

So you could define nodes and for each node for this land graph agent, for example, each node was a LLM that executed a certain action.

So it would be like, either it was answering the question or it was doing research or it was planning or something like that.

And each node would do certain things.

Um, land graph allows you to like kind of simply define controlled edges between those nodes.

So you can say this node directly needs to this node.

Um, but if some, if this happens, then it should lead to this node, just kind of conditional edges.

And, um, it also allows you to save state.

So within, as the agent is running, you can kind of save, uh, just save like each, the results at each, uh, stage.

So like if this node ran, like it's saved in the list.

If this node ran, it's saved in the list.

And that kind of allows you to call back to use this information from this past node to inform your answer for this, this specific node.

And kind of just, it's just like a simple framework to kind of allow you to just build, um, agents effectively.

Um, I don't really, yeah, I think, I think it is hard if you don't have like a good framework to go off of, but we've kind of like built a framework over time.

And actually the agent that's used in the, um, the, the code I put out is, uh, is using that framework.

So if you might find that helpful, if you want to build some LandGraph agents.

Yeah.

Where do you see it all sort of going with different agentic AI and where, where would it look more like a fusion agents or, or collaborative swarms or mesh?

Like, what do you think we might see?

Yeah.

I think I'm a big proponent of the idea that we're going to have like billions of AI agents, just kind of all talking to each other.

Like basically just like a whole AI population.

Like we have a human population.

I think that's probably very likely to happen because at the end of the day, like there's probably going to, I'm skeptical that one model is going to be able to vastly overpower humans.

To the extent that like a bunch of models will be able to.

So like, as you can see in like the typical, like LLM literature, like if you combine, for example, like if you're in your reasoning cycles combined Claude, um, Chatsubut, um, Llama.

And like, if you combine those together in your reasoning, it typically performs better than if you just use only Chatsubut in loops or only Claude in loops.

So kind of having different models trained in slightly different ways is probably going to help increase performance by a significant amount.

So I think we're going to probably have a lot like billions of agents just communicating with each other, maybe at the speed of light, because it's all just information and they don't have to communicate in language like we do as humans.

But, um, yeah, I kind of see that going there.

And then at least in the near future, I think we're like kind of what we were saying, um, we're probably going to see a big rise in using these kinds of formalized frameworks to improve agents.

So like active inference, or there's like a lot of, there's other Bayesian reinforcement learning schemes.

There's typical reinforcement learning schemes.

Um, those stuff are already being used, but I think there's probably going to be a big rise in that as the, um, as the kind of benefits from just purely using like LLM reasoning kind of taper off.

I think we're going to see a big rise in that as well.

That's very interesting.

Like going back to the sort of grass greener, we could see a situation where smaller interpretable active inference type models are using high dimensional interfaces like natural language sharing information.

And the opposite setting where large sort of opaque models are communicating through well-structured narrow channels.

That are active inference, like state spaces and just sending each other that Jason kind of like as a semantically encrypted communication.

You know, right.

It just says three, one, two.

And it's just like, oh, no,
that's not the really insightful point.

Yeah.

It might just be that all these LMs are communicating through.

Yeah.

These narrow cut.

It's kind of like how we communicate too.

It's not like I can't just directly pass my thoughts to you, you know, like ideally if I could, like you would get a lot more information about what I'm thinking, like what I'm feeling, stuff like that.

We have to communicate through words, which is kind of a narrow representation of what our thoughts actually are.

So maybe LLMs have like a similar, you know.

A lot to say about that, but that again comes to the cognitive science topics of communication, semantics, theory of minds.

You know, how cool.

Right.

How cool.

Exactly.

Um, yeah.

What are your next moves with your education and everything?

And then if there's anything else you want to add, you can go for it.

Hmm.

Let's just continue to do, um, continue to do this research, try to expand.

Um, right now, actually what I'm working on is the thing I said about like actually encoding knowledge within the agents, instead of just using some like, like, um, unknown kind of info state modality.

Um, I'm working on that right now, um, kind of using a hierarchical model.

Um, so I think that's like the near future kind of, yeah, I just want to continue working on this whole idea of like combining active inference and LLMs to do as like much interesting things as possible and improve LLMs and active inference at the same time.

Um, one other thing is actually, I thought was really interesting, um, was I was reading the paper on, uh, renormalizing generative models from, uh, Carl Pristen and all of them.

But I think that, that seems really interesting because obviously one of the big problems with active inference is just scale.

It's, it's kind of hard to just keep making more and more and more complex models over and over again.

Like, like LLMs and like typical reinforcement learning have it easy because they just put a policy on it and they just like put a bunch of data in there and then they're good to go.

You know, but we actually have to like painstakingly like build all these generative models and it kind of slows our progress down a lot.

So using these renormalizing generative models, um, to kind of just take away that whole scale problem.

It can just kind of run these group transformations on any, any space of any scale, um, would be pretty useful.

I'm trying to explore, see how I can use that as well.

I think one interesting thing with that is doing it, doing the RGM transformation and more like abstract state spaces.

So like in the paper, they use stuff like, um, like pixels and stuff like that, like physical spatial spaces.

Right.

But maybe doing it on something like language might be a little harder because how exactly do you like, there's no, there's no like exactly pixel equivalent of language.

You know, cause pixels are simple.

You could just say this is a discrete quantity.

The only way you can differentiate this discrete quantities is by RGB values and you're like good to go.

But with language, it's kind of like you have each, it's not like each letter really means anything and putting letters together has different meaning and putting words together has different meaning.

So it's kind of a hard, it's a much harder problem to solve to use the ger, like renormalizing generative models on these language spaces versus just spatial spaces.

So I think that's also an interesting problem, but yeah, kind of just try to think about these things, see what contributions I can make.

Yeah.

Cool.

Yeah.

Great comments from earlier.

Par and Friston et al work on reading as multi-scale active inference.

There's the eye saccade.

It's reducing uncertainty on the letter.

Then letter to letter to reduce uncertainty on the word.

And then they go up to the third level with a narrative, you know, cat ate bird, or was it the bird ate the cat?

So those kinds of things you get priors over letters, transition matrices, all these kinds of things.

And then, but that was, as you put it, painstakingly crafted.

And where could we abstract and have generalized templates so that perhaps an adapter module is crafted to connect a given domain or system of interest to some known interoperable scheme.

And then something like an RGM can take over from there.

Okay.

I'm going to ask one more question from the chat and then that'll be great.

Okay.

All right.

Are you using meta learning aspects in the system to enable self-guided research and are direction opinions guided simply by act-inth?

What was that last part?

Are direction opinions guided simply by act-inth?

Okay.

Yeah.

Yeah.

There's really no meta learning.

The learning rate is stable throughout the entire thing.

Yeah.

The direction opinions are, are like in which, like which search action should be taking or should it, should a search action be taken or should a prompt actually be taking?

That's all purely guided by active inference.

Yeah.

Yeah.

So that was kind of the main point was that the active inference has this formalized structure that can allow for like optimality in terms of like exploration exploitation.

So the active inference agent is purely responsible for picking the actions and determining where to go next.

Yeah.

Epic.

Epic.

Well, thank you very much for joining.

I hope you engage and, and find collaborators and replicators who are interested in this hot topic of active inference and LLMs and come back when you're ready for 95.2.

All right.

Cool.

Okay.

Thank you.

Thank you.

Peace.

Bye.

Bye.

Bye.